Adda: Towards Efficient in-Database Feature Generation via LLM-based Agents(Technical Report)

Anonymous

Abstract

Integrating machine learning (ML) analytics into existing database management systems (DBMSs) not only eliminates the need for costly data transfers to external ML platforms but also ensures compliance with regulatory standards. While some DBMSs have integrated functionalities for training and applying ML models for analytics, these tasks still present challenges, particularly due to limited support for automatic feature engineering (AutoFE), which is crucial for optimizing ML model performance. In this paper, we introduce Adda, an agent-driven in-database feature generation tool designed to automatically create high-quality features for ML analytics directly within the database. Adda interprets ML analytics tasks described in natural language and generates code for feature construction by leveraging the power of large language models (LLMs) integrated with specialized agents. This code is then translated into SQL statements using a predefined set of operators and compiled just-in-time (JIT) into user-defined functions (UDFs). The result is a seamless, fully in-database solution for feature generation, specifically tailored for ML analytics tasks. Extensive experiments across 14 public datasets, with five ML tasks per dataset, show that Adda improves the AUC by up to 33.2% and reduces end-to-end latency by up to 100x compared to Madlib.

ACM Reference Format:

Anonymous. 2018. Adda: Towards Efficient in-Database Feature Generation via LLM-based Agents(Technical Report). In *Proceedings of Make sure to enter the correct conference title from your rights confirmation emai (Conference acronym 'XX)*. ACM, New York, NY, USA, 16 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The growing demand for ML-based analytics, especially for structured data, is undeniable [24, 26, 46, 73]. Applications like stock prediction, fraud detection, and sales forecasting increasingly require ML models tailored specifically for structured data. In response, several DBMSs [23, 29, 41] have integrated ML models directly within their databases. This integration not only streamlines model training and deployment but also introduces SQL extensions that allow users to interact with these models effortlessly. Moreover, it eliminates costly data transfers between databases and external ML platforms, while ensuring compliance with regulatory standards such as GDPR (General Data Protection Regulation), assuming that DBMSs have already met these standards.

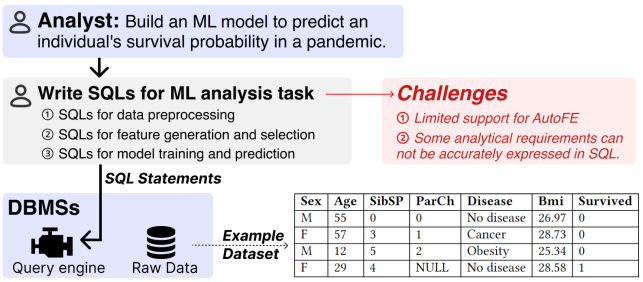

Figure 1: Conducting ML analytics tasks using existing DBMSs presents significant challenges.

Consider an ML analytics task aimed at predicting the probability of an individual's survival during a pandemic, as illustrated in Figure 1. The model is built using a dataset containing sensitive structured data, such as demographics, family members, and health information. As shown in Figure 1, the typical workflow for this task includes data preprocessing, feature extraction, model training, and prediction. Due to regulatory requirements and performance considerations, it is crucial that the entire ML process is conducted within the database, without exporting data to external systems.

It is widely acknowledged that the success of an ML task often depends more on the quality of feature extraction than on the model itself [68]. However, existing DBMS tools like Madlib [23] and PostgresML [41] offer limited support for automatic feature engineering (AutoFE), focusing primarily on model building rather than feature extraction. Feature engineering is a challenging task, and some feature extraction operations are difficult—if not impossible—to express in SQL. For instance, creating a new feature by combining two existing features using a linear equation goes beyond SQL's capabilities. Moreover, defining complex ML analytics tasks, especially AutoFE, in a declarative language like SQL poses a significant challenge for current DBMSs. This often requires extensive human intervention, placing a considerable burden on analysts.

To address these challenges and enable analysts to focus on refining models rather than manually constructing entire analytics workflows, we introduce Adda: an agent-driven in-database feature generation tool designed to automatically create high-quality features for ML analytics tasks directly within databases. Adda bridges the gap between analysts and existing DBMSs by leveraging the power of large language models (LLMs) to interpret analytical requests described in natural language and generate feature construction code. It then utilizes abstract syntax trees (AST) and just-in-time (JIT) compilation techniques to translate this code into SQL statements, providing an efficient, automated in-database solution for feature generation, specifically tailored to ML analytics tasks.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXXXX.XXXXXXX

Given an ML analytics task expressed in natural language, Adda follows a two-stage processing approach. In the first stage, Adda employs a general search framework, enhanced by two specialized agents, to automate the generation of features and corresponding Python code. These agents are carefully designed to harness the power of LLMs, incorporating domain-specific knowledge and contextual insights into the feature construction process. One agent focuses on generating new features and crafting their implementation code, while the other is dedicated to improving the accuracy and reliability of the code. Together, they collaborate to produce and refine high-quality features.

In the second stage, Adda uses a set of predefined operators and conducts AST analysis to translate the code into SQL statements. For Python code that cannot be directly translated into SQL, Adda encapsulates it as user-defined functions (UDFs). Additionally, Adda applies JIT compilation and two optimization strategies to generate efficient, compiler-friendly SQL statements. This end-to-end approach provides a complete in-database feature generation solution, resulting in significant performance improvements.

We implement Adda on PostgreSQL [60] and conduct extensive experiments on 14 public datasets, with five ML analytic tasks for each, to evaluate its efficiency and effectiveness. Experiment results demonstrate that Adda improves the AUC by up to 80.7% and reduces the end-to-end latency by up to 20x compared to CAAFE [24]; and Adda achieves up to 33.2% for the AUC improvement and 100x for the end-to-end latency reductions compared to Madlib [23].

To summarize, our key contributions are as follows:

- We present Adda, an agent-driven in-database feature generation tool, which adopts a two-stage processing approach to generate high-quality features for ML analytics tasks.
- We propose an agent-based feature generation framework that utilizes a general search framework, integrating two specialized agents to harness the full potential of LLMs.
- We introduce an efficient in-database feature computation method that translates Python code into compiler-friendly SQLs using the JIT compilation and two UDF optimization.
- Experiments conducted on various ML tasks and public datasets show that Adda achieves significant ML model performance improvements and end-to-end time reductions.

2 An Overview of Adda

In this section, we give an overview of Adda and formally define its automatic feature engineering problem.

2.1 System Architecture

Numerous studies [7, 14, 28, 68] have emphasized the critical role that feature generation plays in determining the overall performance of ML models. However, existing DBMSs offer limited support for AutoFE. To address this gap, Adda focuses on efficiently generating high-quality features directly within the database environment. To achieve this, we developed a two-phase processing strategy, as shown in Figure 2. When an ML analytics task is provided in natural language, Adda begins by utilizing an agent-based feature generation component to automatically construct features and generate the corresponding Python code. In the next phase, Adda's in-database feature computation component translates the Python code into SQL statements. This approach delivers a fully automated, in-database feature generation solution. In the following sections, we will explore the details of these two components.

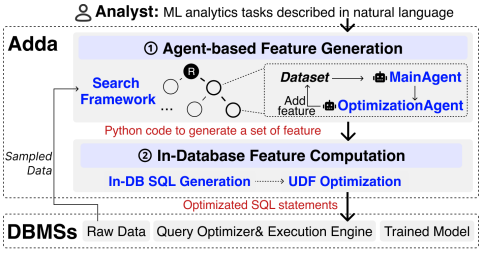

Figure 2: System architecture of Adda.

(1) Agent-based Feature Generation. Adda leverages the natural language understanding capabilities of LLMs to interpret analysts' requests. It first extracts a sample dataset from the database to construct features. The sample dataset not only mitigates the risk of data leakage but also enhances the efficiency of feature construction. To generate new features, a tree search framework with two specialized agents is employed: the main agent and the optimization agent. The main agent follows a retrieval-augmented-generation (RAG) process to propose new features and generate their corresponding Python code, while the optimization agent improves the accuracy of the generated code by breaking down complex features into simpler components. Working iteratively within the tree structure, these two agents collaborate to identify an optimal set of features tailored to the specific ML analytics task.

(2) In-Database Feature Computation. We translate the Python code for feature generation into SQL statements by utilizing a set of predefined operator types and performing AST analysis during the in-database SQL generation process. For Python code that cannot be directly converted into SQL, we encapsulate it as UDFs. Additionally, we provide C versions for commonly used Python code to facilitate JIT compilation. To enhance efficiency, we use Common Table Expressions (CTEs) to combine multiple SQL statements into a single, larger query, reducing unnecessary persistent storage overhead. Furthermore, we introduce two optimization strategies—Merge UDFs and Shrink UDFs—to streamline UDFs. This approach produces efficient, compiler-friendly SQL statements, enabling seamless model generation without sacrificing performance.

Discussion. Instead of generating SQL statements directly, our Adda first generates Python code to construct features and then translates the code into SQL statements. This approach is necessary because certain features are challenging, if not impossible, to express in SQL. For instance, creating a new feature by combining two existing features using a linear equation goes beyond SQL's capabilities. Moreover, the AutoFE process often involves iterative tree searches, feature computations, and verifications, which are too complex to be adequately expressed in SQL. If we generate SQL statements directly, we would face not only a loss in model performance because of the limitations of SQL expressed features but also a loss in efficiency due to additional data transfers between DBMS systems and Python platforms. This inefficiency arises because we would need to execute SQL within a database to generate new features

and then transfer these features out of the database to the agent for further feature generation, even when working with sampled data.

2.2 Automatic Feature Engineering Problem

Adda is dedicated to automating the feature engineering process for ML analytic tasks involving structured data. We formally define our problem as follows:

Let $\mathcal{D} = (F, Y)$ be a dataset, where (i) F is a set of original features, $F = \{f_1, \cdots, f_n\}$; (ii) Y is a target attribute for prediction or classification. Considering a downstream ML model L, we aim to find an optimal feature set $\hat{F}$ that maximize the performance of L. Features in $\hat{F}$ are generated by agents leveraging the power of LLMs to incorporate domain knowledge into feature generation. Given a dataset $\mathcal{D} = (F, Y)$ and a downstream ML model L, the feature engineering problem is defined as follows:

$$\arg\max_{\hat{F} \subseteq \mathbb{F}} \mathcal{L}(L(\hat{F}, Y)) \tag{1}$$

where (i) $\mathcal{L}$ is a performance metric for an ML model L, such as the AUC or F1 score; (ii) $\hat{F}$ is an optimal feature set that maximizes the performance metric of L. (iii) $\mathbb{F}$ is an infinite set of features, denoted as $\mathbb{F} = F_\infty$, where each feature is derived from the transformation of original features F (i.e., $F_0 = F$ and for each subsequent set, $F_{n+1} = F_n \cup \{f_{new} | \forall f_1, ..., f_k \in F_n, f_{new} = o(f_1, ..., f_k)\}$.) The transformation o is devised by LLM agents, encompassing any viable computation to derive novel features from the existing feature set.

Table 1: Notations used in this paper.

Notation	Meaning
$\mathcal{D}, Y$	a dataset and a target attribute.
f, F	one feature and a set of features.
c, C	a feature's execution code and a set of execution code.
d, D	a feature's description and a set of descriptions.
$L, \mathcal{L}$	an ML model and its performance metric.
$\mathcal{U}$	the node utility for a feature set.
M	the cyclomatic complex of a code.

3 Agent-based Feature Generation

In this section, we present our agent-based feature generation framework, which leverages the power of LLMs to incorporate open-world domain knowledge into the feature generation process.

3.1 Search Framework with Two Agents

The agent-based approach has become a prominent solution for addressing specific tasks by leveraging the capabilities of LLMs [15, 25, 74]. In this approach, agents are carefully designed to interact with both the LLM and the external environment to accomplish their objectives. In Adda, we adopt this methodology to find an optimal feature set, $\hat{F}$, by incorporating two agents into a search framework. The two agents are: (i) the main agent, responsible for generating new features, and (ii) the optimization agent, which enhances the accuracy of the generated features by breaking down complex ones into simpler components. Both agents collaborate, utilizing the LLM's capabilities to iteratively generate and refine features.

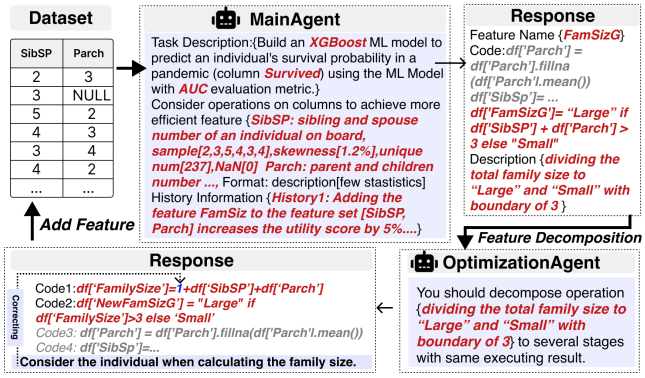

Figure 3: A collaboration example of the two agents.

Figure 3 illustrates the collaborative process of the two agents using the task with XGBoost model and AUC metric. To generate a new feature, the two agents collaboratively leverage the semantic understanding capabilities of an LLM to incorporate domain knowledge into feature generation. The main agent takes in all existing features and, with the aid of an LLM, selects SibSP and Parch to generate a new feature FamSizG along with its description and code. This new feature assesses the survival possibility of individuals according to their family size, dividing families into two groups: large or small. Subsequently, the optimization agent refines the new feature by decomposing it, utilizing the capabilities of the LLM. It takes the description and code of the new feature and breaks it down into two simpler features: FamilySize and NewFamSizG. First, it calculates the size of each family, then groups families based on size. In this process, the optimization agent corrects an error in the code generated by the main agent, where the individual was initially omitted when calculating family size. Additionally, Adda suggests some other meaningful features, such as SickNum, which represents the number of sick individuals within a family, and ElderlyNum, indicating the number of elderly people within a family.

We utilize this collaborative process to create features and identify an optimal feature set, $\hat{F}$, through a search framework, as detailed in Algorithm 1. The core idea behind Algorithm 1 is to iteratively explore a tree, where new features are generated at each step by the two agents. These newly generated features are then considered for the construction of additional features, incrementally enriching the existing feature sets and expanding the tree with new nodes. Each node in the tree represents a feature set, and each directed edge from a parent node, F_{par}, to a child node, F_{child}, represents the generation of new features, F_{new}, from F_{par}, which are then added to F_{child} (i.e., $F_{child} = F_{par} \cup F_{new}$). The root node of the tree corresponds to the original feature set, F.

Inspired by the commonly used upper confidence bound (UCB) [6] solution, our algorithm for node expansion in Algorithm 1 also balances the trade-off between exploitation and exploration. To achieve this, we select the node that results in high performance of the downstream ML model (i.e., exploitation) and also has little information to inform future choices (i.e., exploration) to expand.

Algorithm 1: Agent-based feature generation

Input: F, Y, L
Output: A set of new features $\hat{F}$ and their code $\hat{C}$ for generating them.

1 $Q = \{(F, \emptyset, \mathcal{U}(F, Y, L))\}$;
2 **for** $i \leftarrow 1$ **to** s **do**
3 $\quad (F, C,) \leftarrow$ POP_MAX_$\mathcal{U}(Q)$;
4 $\quad$ **for** $j \leftarrow 1$ **to** t **do**
5 $\quad\quad f_j, c_j, d_j \leftarrow$MainAgent$(F, Q)$;
6 $\quad\quad F_j, C_j \leftarrow$OptimizationAgent$(f_j, c_j, d_j)$;
7 $\quad\quad F_{child} = F_j \bigcup F$; $C_{child} = C_j \bigcup C$;
8 $\quad\quad Q = Q \bigcup \{(F_{child}, C_{child}, \mathcal{U}(F_{child}, Y, L))\}$;
9 $\quad Q = Q \bigcup \{(F, C, \mathcal{U}(F, Y, L))\}$;
10 Change all $\mathcal{U}$ value to their corresponding $\mathcal{L}$ value in Q;
11 $(\hat{F}, \hat{C},) \leftarrow$POP_MAX_$\mathcal{L}(Q)$;
12 **return** $\hat{F}, \hat{C}$;

In Adda, we define the node utility $\mathcal{U}$, which takes both exploitation and exploration into account so as to guide node expansion during the search. The node utility is

$$\mathcal{U}(F, Y, L) = \mathcal{L}(L(F, Y)) + w \cdot \sqrt{\frac{ln(v_0)}{v_F}} \tag{2}$$

where (i) $\mathcal{L}$ is a performance metric (i.e., the AUC or the F1) of L as mention in Section 2; (ii) v_0 is the sum of visited number for all nodes; (iii) v_F is the visited number of current node; (iv) w is a weight factor to achieve the trade-off between exploitation and exploration, and we set it to $\sqrt{2}$ in our experiments [6]. Additionally, we arm each node with a triplet, which includes its feature set F, the code C for generating new features in F and its utility $\mathcal{U}$, in Algorithm 1. The details of Algorithm 1 is as follows.

Given a dataset $\mathcal{D} = (F, Y)$ and an ML model L, Algorithm 1 returns an optimal feature set $\hat{F}$ together with its corresponding code set $\hat{C}$ for generating new features in $\hat{F}$.It first initializes a priority queue Q with the root triplet $(F, \emptyset, \mathcal{U}(F, Y, L))$ because the initial feature set is F and the initial code is empty (line 1). The higher node utility, the higher priority in Q. Next, Algorithm 1 iterates s steps and selects the highest utility node (i.e., POP_MAX_$\mathcal{U}$) to expand t children nodes for each step (lines 2∼line 9). When expanding a child node, Algorithm 1 invokes the MainAgent(F, Q) to generate a child feature f_j and its corresponding code c_j and description d_j (line 5), calls the OptimizationAgent(f_j, c_j, d_j), which refines the generated feature f_j by decomposing the complex logic feature into simpler ones (line 6), and combines the parent feature set F and code C together with the new generated feature set F_j and code C_j, respectively (line 7). The new generated child node is added into Q (line 8). After expanding t children nodes, Algorithm 1 updates the parent node by recomputing its utility as v_0 and v_F change (line 9). Finally, we select and return the node in Q with the maximal $\mathcal{L}$ score without considering the exploration item (line 10∼line 12).

3.2 MainAgent: Generating One Feature

The main agent is responsible for proposing new features for analytical tasks. Generally, it follows a retrieval-augmented generation (RAG) procedure, but with a unique enhancement: it maintains a priority queue, Q, as an internal state to track the history of feature generation. This queue records all generated feature sets along with their respective utilities. The main agent is designed to explore the dataset, aiming to discover new potential features with rich semantics and utility based on the original features. It leverages an LLM's feature generation capability by interacting with the LLM through two types of prompts: static and dynamic.

The static prompt, which remains constant throughout the feature generation process, includes (i) a description of the analytical task, specifying the ML model, target attribute, and performance metric, (ii) a schema for each feature, detailing its description and data type, and (iii) sampled data for each feature, along with statistics such as skewness, distribution, number of unique values, and proportion of NaN values. For example, in Figure 3, the task description, part of the static prompt, specifies that the ML model is XGBoost, the target attribute is Survived, and the performance metric is AUC. The static prompt also includes detailed information about each feature. For instance, the feature SibSp, which indicates the number of siblings and spouses an individual has, exhibits a skewness of 1.2%, contains 237 unique values, and has no NaN values. The static prompt informs the agent about the task at hand and provides necessary context—such as each feature's schema and sampled data—to enhance the accuracy of feature generation code.

The dynamic prompt is generated by leveraging the RAG procedure to query nodes in Q that are similar to the currently selected node. These similar nodes, serving as the dynamic prompt, are provided to the main agent as historical experience to enhance the quality of the newly generated features. By learning from this historical experience, the agent can identify features that are likely to improve or degrade model performance, thereby generating new features with a high likelihood of enhancing performance. Each time the main agent generates a new feature, it queries Q for the necessary dynamic prompts. An example historical experience, acting as the dynamic prompt for the main agent in Figure 3, describes the utility gain achieved by adding the feature FamilySize to the existing feature set [SibSP, Parch]. Further details regarding the RAG-enhanced dynamic prompt are elaborated in the subsequent Section 3.2.1.

With the aid of the above prompts, the main agent first responds with a feature name for the newly generated feature f, a semantic description d of f, and its computation method, which specifies the operators and the involved features necessary for the computation. Subsequently, the main agent integrates these responses along with the prompts to form the context for the LLM to generate the execution code c for f. The code c is verified using the sample dataset. In Figure 3, the main agent takes in both the static and dynamic prompts and generates a new feature FamSizG, along with its feature description d and execution code c, which includes the necessary data preprocessing code. Noting that the semantic description d of a newly generated feature f enhances the interpretability of AutoFE by incorporating real-world insights from LLMs.

3.2.1 *RAG-enhanced Dynamic Prompt.*

As mentioned before, the dynamic prompt is constructed in accordance with the RAG procedure. Specifically, due to the input context limitation of an LLM, we query the top-k nodes from the

priority queue Q that are most similar to the current node. The selection of k depends on the LLM's context length limitation. We continue encapsulating the most similar nodes into the prompt until the context limitation of an LLM is reached. Then, these similar nodes, serving as the dynamic prompt, are provided to the main agent to augment its capability for feature generation. For each feature generation, the main agent queries Q to retrieve the necessary dynamic prompts. The similarity between nodes is primarily evaluated based on their feature sets, leveraging a product of two metrics: SemantSim and StructSim, which consider the similarity from the semantic and structure aspects, respectively.

$$\text{Sim}(F_1, F_2) = \text{SemantSim}(F_1, F_2) * \text{StructSim}(F_1, F_2)$$

SemantSim: The semantic similarity between two features is calculated by their descriptions' similarity using an embedding model. General text embedding models, such as BGE [69] and GTE [44], fail to adapt effectively to the specific feature generation task. Therefore, we offline fine tune an existing embedding model using the contrastive learning before feature generation. Specifically, we first collect dozens of datasets from OpenML, UCI repository and Kaggle, and then obtain feature names and their corresponding descriptions from the dataset. In contrastive learning, each data point is represented as $[d, d^+, (d_1^-, \ldots, d_n^-)]$, where (i) d is a feature description from one dataset mentioned above, (ii) d^+ is a positive sample of d obtained using an LLM to re-generate a similar description of d, and (iii) $(d_1^-, \ldots, d_n^-)$ is a set of n negative samples, which are randomly sampled from other datasets. Contrastive learning aims at maximizing the similarity between positive pairs while minimizing that between negative pairs. The infoNCE [64] loss is adopted here for finetuning. Finally, we use the fine tuned model E to compute the semantic similarity between two features by first embedding their descriptions into vectors and then calculating their cosine similarity. Therefore, the semantic similarity between two feature sets can be computed by:

$$\text{SemantSim}(F_1, F_2) = \frac{1}{|F_1| * |F_2|} \sum_{d_1 \in F_1} \sum_{d_2 \in F_2} \cos(E(d_1), E(d_2)) \quad (3)$$

StructSim: We compute the structural similarity between two feature sets by analyzing the generation relationship maintained in them. Given that all feature sets in Q derive from the same original feature set, we focus solely on the generated features in a feature set when measuring the structural similarity. Specifically, we first convert each generated feature within a feature set into a tree that represents its generation lineage, using the original features as references. These generated features in a feature set collectively form a forest. We then convert the forest into a single tree by recursively merging identical nodes from the root to the leaves at each level. Finally, we compute the structural similarity between two feature sets using the tree edit distance [38, 72], which is the minimal number of node operations, such as replacements, deletions, and insertions, required to transform one tree to another. For instance, consider an original feature set $F = \{f_1, f_2, f_3\}$, and two feature sets $F_1 = F \bigcup \{f_4\}$ where $f_4 = f_1 + f_2$, and $F_2 = F \bigcup \{f_5\}$ where $f_5 = f_1 + f_3$. The tree edit distance between the converted trees of F_1 and F_2 is 2. After calculating the structure similarity between the current feature set and each in Q, we normalize these values to the range $(0, 1)$ using the sigmoid function.

3.3 `OptimizationAgent`: Improving a Feature by Decomposition

After generating the execution code c for a feature f by the main agent, an optimization agent is designed to improve the correctness of the code. A key challenge the optimization agent faces is to generate correct code features while maintaining the performance of downstream ML tasks. The reasons are as follows. (i) The main agent tends to search for complex features, as they often play a crucial role in enhancing the performance of ML tasks [10, 75]. (ii) But the complex logic within the code is easily out of the generation capability of an LLM [47, 59], resulting in exceptions and the necessity for code regeneration. A naive solution is to repeatedly invoke the main agent to regenerate a feature until the code is error-free. However, it bypasses complex logic features and chooses simpler ones suffering from the performance loss of ML tasks.

To guarantee both the correctness of the generated code and the performance of ML tasks, our optimization agent adopts a decomposition approach, which breaks a complex logic feature f into multiple simpler steps without altering its semantic meaning d proposed by the main agent. The main agent can propose new features involving any computations, and thus, merged features can also be proposed by the main agent. In Figure 3, the optimization agent decomposes the complex feature FamSizG into two simpler steps by first computing the size of each family (i.e., FamilySize) and then grouping families based on their size (i.e., NewFamSizG). Specifically, we recursively leverage the optimization agent to divide a complex feature into a collection of simpler ones until the generated children features match its parent feature, indicating that further decomposition is unnecessary. Additionally, a rollback mechanism is introduced in our decomposition approach to stop early. It inspects the complexity of the code required to generate a feature and stops decomposition if further breaking down the feature would increase the code's complexity.

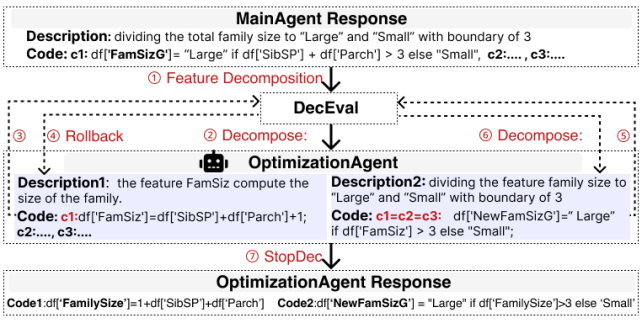

Figure 4: Feature decomposition with the optimization agent.

Figure 4 illustrates the approach of feature decomposition with the optimization agent, exemplified by the breakdown of the feature FamSizG. The approach takes in the description and three repeatedly generated code c_1, c_2, c_3 of the feature FamSizG provided by the main agent. Our approach recursively leverages the `DecEval` function to assess if a feature needs decomposition, activating the optimization agent for this purpose when necessary, as depicted in Figure 4 steps ① to ⑦. The `DecEval` function decides to decompose

the feature FamSizG (②). The optimization agent then decomposes it into FamilySize and NewFamSizG. These two features are then evaluated by the DecEval function; it decides to rollback for FamilySize and not to decompose for NewFamSizG(③∼⑥). Our approach stops decomposition and returns the two features and their corresponding code. A comprehensive description of our feature decomposition approach is detailed in Algorithm 2.

Algorithm 2: The decomposition of a feature.

Input: A feature f together with its corresponding code c and feature description d.
Output: A set of decomposed features F and their code C.

1 $H = \{(f, c, d)\}, H' = \emptyset$; /* Initialization. */
2 **while not** StopDec(H, H') **do**
3 $H' = H, H = \emptyset$;
4 **for** $(f, c, d) \in H'$ **do**
5 $H_{dec} = \{(f, c, d)\}$;
6 **if** DecEval(f, c, d) **then**
7 $H_{dec} \leftarrow$OptimizationAgent (f, c, d);
8 **if** Rollback(H_{dec},c) **or** StopDec(H_{dec},c) **then**
9 $H_{dec} = \{(f, c, d)\}$;
10 $H = H \cup H_{dec}$;
11 $F, C \leftarrow$ get the set of features and their code from H;
12 **return** F, C

Algorithm 2 takes in a feature f along with its code c and description d, and returns a set of decomposed features F and their code C. In Algorithm 2, we first initialize a feature triplet set H with the input feature f, its code c, and its description d to maintain the feature information. Meanwhile, a temporary set H' is initialized as empty (line 1). Subsequently, we recursively divide a complex feature into a collection of simpler features (line 2∼line 10). During each iteration, we first evaluate whether to decompose a feature f or not using the DecEval function (line 6). If we decide to decompose f, we employ the OptimizationAgent to divide the feature f into a collection of simple features H_{dec} by leveraging the capabilities of LLMs (line 6∼line 7). Algorithm 2 stops decomposition when the generated features set matches the original one (line 2). We next explain several specific functions used in Algorithm 2.

DecEval (line 6): We evaluate whether a feature needs further decomposition through two criteria. One is the self-consistency of a feature's code generated by the main agent. The self-consistency theory [65] in LLM is that the greater the consistency among multiple responses generated by an LLM for the same input, the higher the confidence in the final majority response, which theoretically ensures the self-consistency of a feature's code. Specifically, we generate m response code $C = \{c_1, c_2, \cdots, c_m\}$ for a feature f, verify these code on the sample data and compare their results. We choose the one $\hat{c}$ with maximal identical results among C, as $\hat{c}$ is more confidential based on the self-consistency theory [65]. If the maximal number of identical results exceeds 2/3 of the total number of the generated code m, we consider the code $\hat{c}$ to be reliable. Otherwise, $\hat{c}$ is unreliable. The number of 2/3 is from the Byzantine Consensus theory [9] that at least $3p + 1$ responses are needed to tolerate p faulty responses in a single round of communication. In Figure 4, we repeatedly generate three code c_1, c_2, c_3 for the feature FamSizG and examine their results on a sample data, which is maintained as prompt. We find that the results of these code are different from each other. Thus, the code for FamSizG is unreliable because 1/3 is less than 2/3. While the code for NewFamSizG is reliable because the three generated code are the same and 3/3 exceeds 2/3.

Another is the McCabe's cyclomatic complexity M [49] for a feature's code c. Cyclomatic complexity is a metric in Software Engineering to compute the complexity of logical loop in code by measuring the number of linearly independent paths in it [49]. Here it is used to evaluate the complexity of the logic in c. Specifically, we analyze the control flow graph (CFG) of code c and compute the cyclomatic complexity of c by using $M = E - N + 2$, where E represents the number of edges in the CFG and N represents the number of nodes. For example, the cyclomatic complexity of a GROUPBY function, which divides rows into two category, is 2 where $E = 4$ and $N = 4$. More details for computing M can be found in [49]. In fact, the code generated by the main agent operates on DataFrames using Pandas Library functions, a predefined cyclomatic complexity of these functions (e.g., M(GROUPBY)=2, M(AVG)=2) is introduced to facilitate the computation of the cyclomatic complexity for a feature. Consider a feature with AVG(GROUPBY(df['sex'])) to calculate the average salary for each gender, its cyclomatic complexity M= M(GROUPBY) + M(AVG)−1 = 2 + 2 − 1 = 3 where 1 is the reduced cyclomatic complexity by merging the CFG of the GROUPBY function together with that of the AVG function.

Finally, self-consistency and cyclomatic complexity of a feature are evaluated to determine whether further decomposition is necessary. Specifically, if the code c for a feature f is unreliable or its cyclomatic complexity exceeds 10[49], DecEval returns true. Otherwise, it returns false.

OptimizationAgent (line 7): This agent decomposes a complex feature f into a collection of highly likely logically simpler features F_{dec} by first dividing the feature's description d into a group of sub-descriptions $D = \{d_1, \cdots d_m\}$ suggested by LLM. This is achieved by specifying simple prompts, such as 'please split d into multiple steps', to ask an LLM to decompose d. Then we generate executable code $C = \{c_1, \cdots c_m\}$ based on these sub-descriptions using the main agent to generate m simpler features $F_{dec} = \{f_1, \cdots f_m\}$. In Figure 4, the optimization agent first decomposes the description of FamSizG into two sub-descriptions, and then generates their corresponding code using the main agent, thereby constructing FamilySize and NewFamSizG. Noting that our decomposition approach do not require features to be linearly decomposable as aggregating decomposed features is value passing. For example, $avg(norm(a + b))$ can be decomposed into $avg(norm(x))$ and $x = a + b$. Aggregating decomposed features is just passing the value $a + b$ of feature x to $avg(norm(x))$.

Rollback (line 8): We observe that there exist some simple logic features but with vague descriptions. For example, the description of FamilySize, which computes the size of a family, is vague, because the table contains several columns indicating family members, such as brothers and siblings. The code of these features illustrates low cyclomatic complexity but is easily to be unreliable because of vague feature descriptions. The DecEval function decides to decompose

Table 2: Operator types in Adda

Operator Type	Operator Description	Example Python Code[1]	SQL Statement
Unary	Conduct arithmetic computation on a feature	df['f_4'] = (df['f_1']/2).astype(int)	SQL: CREATE TABLE T1 AS SELECT (CAST (f_1/2 AS INT)) AS f_4, * FROM SRC;
Numerize	Map a non-numeric feature into an integer feature	df['f_4'] = LabelEncoder.fit_transform(df['f_1'])	SQL: CREATE TABLE T1 AS SELECT (CASE WHEN f_1='op1' THEN 0, CASE WHEN f_1='op2' THEN 1$\cdots$) AS f_4, * FROM SRC;
Drop	Drop some feature from feature set	df = df.drop(['f_1', 'f_2'])	SQL: CREATE TABLE T1 AS SELECT f_3 FROM SRC;
Multi-element	Generate a new feature by arithmetic computation on several input features	df['f_4'] = df['f_1'] + df['f_2'] / df['f_3']	SQL: CREATE TABLE T1 AS SELECT f_1 + f_2/f_3, * FROM SRC;
Normalize	Scale a feature using MinMaxScaler, StandardScaler or RobustScaler	df['f_4'] = StandardScaler.fit_transform(df['f_1']))	SQL: CREATE TABLE T1 AS SELECT (f_1 - AVG(f_1)) OVER ())/STDDEV(f_1) OVER() AS f_4, * FROM SRC;
FillNaN	fill the NaN values in a feature with its mean, mode, median or a constant value.	df['f_4'] = df['f_1'].fillna(df['f_1'].mean(), inplace=False)	SQL: CREATE TABLE T1 AS SELECT (CASE WHEN f_1 IS NULL THEN (SELECT AVG(f_1) FROM DF WHERE f_1 IS NOT NULL) ELSE f_1) AS f_4, * FROM SRC;
Discretize	Convert a feature with continuous values into discrete labeled groups	df['f_4'] = pandas.cut(df['f_1'], bins=3, label=['low','middle','high'])	**UDF(C/Python):** CREATE TABLE T1 AS WITH T1 AS(SELECT CUT/TRAIN/PREDICT/UDF/UDF(array[SELECT row(f_2, f_1) FROM DF], array['low', 'middle', 'high']) as cut FROM SRC), SELECT (unnest(cut)).* from DF1;
UDF	Encapsulate a Python code snippet as a UDF	df['f_4'] = df['f_1'].apply(func, params)	

[1] We present some Python code examples with an original feature set {f_1, f_2, f_3}.

these features. But decomposing them increases the cyclomatic complexity of the code for sub-features resulting in increasingly logical complexity and error-prone. Based on this observation, a `Rollback` mechanism is introduced to stop decomposition early. Specifically, the rollback mechanism inspects the cyclomatic complexity of the code required to generate a feature. After decomposing a feature f and generating its corresponding sub-features F_{dec}, if the cyclomatic complexity of the code for producing F_{dec} is equal to or greater than that of the code for producing f, we rollback to f and stop decomposing f. Thus, decomposing a feature can not increase its cyclomatic complexity. In Figure 4, the feature FamilySize is rollback.

`StopDec` (line 8 & line 11): Besides the `Rollback` strategy, a `StopDec` function is introduced to terminate our iterative decomposition in Algorithm 2. Our `StopDec` function compares the generated column values of a feature f to that of its decomposed features F_{dec}. Specifically, if the column values of executing f equals to that of sequentially executing all decomposed features in F_{dec}, indicating that further decomposition is unnecessary, we stop our iterative decomposition in Algorithm 2. In Figure 4, as FamilySize is rollback and NewFamSizG is not further decomposed, the subsequent decomposition iteration for FamilySize and NewFamSizG returns the same features. This satisfies the `StopDec` condition. As a result, Algorithm 2 terminates the decomposition process and returns the decomposed features FamilySize and NewFamSizG.

4 In-Database Feature Computation

In this section, our Adda efficiently constructs features within databases by converting their code into SQL statements and leveraging the sophisticated optimization techniques inherent in databases.

4.1 In-database SQL Generation

Performing ML analytic tasks, including feature generation, model training, and prediction, entirely within databases delivers several advantages. Firstly, it eliminates the frequent and heavy data transfers between Python environments and database systems, particularly for data-intensive feature generation, thus accelerating the overall training and iterative process. Secondly, it harnesses databases' native capabilities, such as query optimization, horizontal scaling, and reliability, ensuring high performance and reliability for model training and prediction tasks. Last but not least, it ensures compliance with regulatory standards, such as the GDPR, with assumptions that DBMSs have already met these standards.

To convert the entire feature generation Python code from Section 3 into SQL statements, we sequentially translate the Python code for each feature into a SQL statement. Each feature's SQL statement ingests the prefix features via the SELECT * statement and outputs the feature along with its prefix features using the CREATE TABLE statement. To generate the SQL statement for a feature, we introduce a group of commonly used operator types together with their descriptions, Python code examples, and SQL statements in Table 2. It is worth noting that features generated by the two agents may involve various computations. Some of these features may include sophisticated Python code, such as a linear equation on two features and the Discretize operator in Table 2, which exceeds the scope of operator types listed in Table 2. The UDF characteristic supported in SQL is essential to bridge the gap. We encapsulate this complex Python code within a UDF and registered in a database for execution. We also provide C version for some common Python code and utilize the JIT compilation technique to enhance efficiency. Meanwhile, we retain supplementary information for each UDF, including the feature names involved in computing the feature and the output feature name, to enable further optimization of these UDFs. Additionally, two specialized UDFs: TRAIN and PREDICT, are introduced to encapsulate Python code for model training and prediction, respectively.

Figure 5 illustrates the process of translating a feature's Python code into a SQL statement. Consider an example Python code df['f_4']= StandardScaler.fit_transform(df['f_1']). We convert this code into a Python AST and find that it matches that of the `Standard Scaler Normalization` operator in Table 2. Next, we translate the code to the SQL statement: CREATE TABLE T1 AS SELECT (f_1 - AVG(f_1)) OVER() / STDDEV(f_1) OVER() AS f_4, * FROM SRC. This translation is achieved by leveraging the matched operator `Standar-`

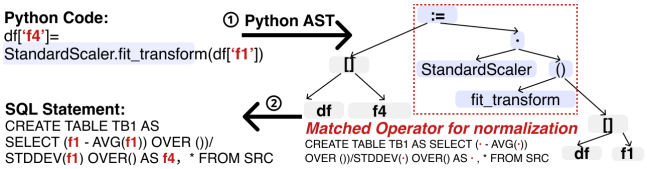

Figure 5: Translating Python code to SQL statements.

dScaler Normalization's SQL template: CREATE TABLE T1 SELECT (· - AVG(·)) OVER() / STDDEV(·) OVER() AS ·, * FROM SRC, and populating the template with the input and output feature names: f_1 and f_4, respectively. The SQL statement ingests prefix features from SRC via SELECT * and stores the new feature f_4 and its prefix features to T1 through the CREATE TABLE T1 statement.

In general, for an optimal feature set $\hat{F}$ with its Python code $C = \{c_1, \ldots, c_m\}$, the process of generating equivalent SQL statements is presented as follows. For each Python code c_i in C, we first parse it into a Python AST and then check if this Python AST matches any of the Python ASTs for operator types in Table 2. If matching, we generate its SQL statement using the matched operator type's SQL statement; otherwise, we encapsulate the Python code as a UDF. Finally, we translate the entire C into a series of SQL statements to generate an optimal feature set $\hat{F}$ within a database.

After generating the SQL statements for feature construction in an ML analytic task, we append a TRAIN or PREDICT UDF to the end of these SQLs for model training or prediction, respectively. In Adda, a trained model is serialized into bytes or strings and stored in a table. When needed, the model is deserialized to conduct predictions. The complete set of SQLs for the analytic task, including feature generation, model training, and prediction, is then converted into a large SQL using CTEs without creating intermediate tables. In contrast to Madlib, which materializes each operator's results, our Adda exploits the database's native Volcano Model to avoid unnecessary persistent storage overhead caused by the CREATE TABLE statements. It is worth noting that although our Adda handles model training on a single table, the training of ML models involving multiple tables can be streamlined by joining these tables into a single one, which can then be fed into Adda for processing.

4.2 UDF Optimization

In our experimental evaluation of SQL statements generated for ML analytic tasks, we found that embedded UDFs are the primary contributors to performance bottlenecks. This occurs because the database optimizer treats UDFs as black boxes, unable to discern their internal logic, unlike non-UDF SQL statements, whose unnecessary features derived from SELECT * are automatically filtered by the database's Volcano Model during execution. Additionally, for each UDF within the SQL statements, in addition to the computational cost of executing its internal logic, there are additional costs associated with wrapping and unwrapping features between the database data format and the C/Python data format. This overhead is incurred on every UDF invocation, with all its input features being wrapped and unwrapped, irrespective of their actual use in the computation. To reduce the wrapping and unwrapping costs for UDFs within SQL statements, our Adda introduces two strategies: Merge UDFs and Shrink UDFs.

Merge UDFs. In a series of SQL statements, those that are adjacent and contain UDFs are combined into a single statement by merging their UDFs' inner code into one UDF, thereby ensuring that the UDF is invoked only once and that all its input features are subject to wrapping and unwrapping processes only once.

Shrink UDFs. For a feature generated by a UDF, all of its prefix features are wrapped and unwrapped when the UDF is invoked, regardless of whether a prefix feature is utilized in the computation. Our Adda divides all prefix features into two groups leveraging the supplementary information for the UDF retained when encapsulating the Python code into the UDF as mentioned in Section 4.1. One group is prefix features involved in the feature's computation, denoted as F_{inv}. Other prefix features are belong to another group, denoted as $F_{\overline{inv}}$. For a UDF with prefix features F_{inv} and $F_{\overline{inv}}$, it has two possible plans: UDF($F_{inv} \cup F_{\overline{inv}}$) and JOIN(UDF($F_{inv}$),$F_{\overline{inv}}$), which conduct the UDF computation by only pass the involved prefix features F_{inv} and then join the UDF's output with $F_{\overline{inv}}$. A cost estimation method is introduced to choose one of them.

Specifically, let C_u and C_j be the cost of a UDF operator and a JOIN operator, respectively. As the UDF computation logic in these two plans are the same, the UDF computation cost is not considered in C_u. The cost of these two plans are $C_u(F_{inv} \cup F_{\overline{inv}})$ and $C_u(F_{inv}) + C_j(F_{inv}, F_{\overline{inv}})$. Let u and $\hat{u}$ be the cost of wrapping and unwrapping one byte in a UDF operator, respectively. Both of them is computed by measuring the cost of wrapping and unwrapping one byte in a UDF using a sample of data. Thus, the UDF's cost for a feature set F is $C_u(F) = |F| * u + (|F| + |F_{new}|) * \hat{u}$, where $|\cdot|$ is the number of bytes for a feature set on one row. We use the hash join [54] to compute C_j. After estimating each plan's cost, our Adda chooses the minimal cost plan.

5 Discussion

LLMs in AutoFE. Numerous studies have highlighted the potential of LLMs in AutoFE [21, 22, 24, 46]. Specifically, [21, 22] demonstrate that features generated by LLMs can significantly enhance the performance of ML models. Meanwhile, [24, 46] show that LLMs can extract meaningful and effective features by leveraging their real-world comprehension capability. Additionally, the experiments in Section 6.6.2 demonstrated that features generated by LLM agents were both high-quality and interpretable. Despite these advancements, the usage of LLMs in AutoFE inevitably incurs additional costs in terms of time and expense, as evaluated in Section 6.6.1. Moreover, the reasoning processes of LLMs remain relatively opaque, functioning as a "black-box".

The Usage of LLMs. There are two mainstream methods for utilizing LLMs: one is to access remote LLMs through API calls, and the other is to deploy LLMs locally. When using API calls, prompts are sent to a remote server, and responses are received back. This method allows users to easily access multiple remote LLMs, particularly when privacy requirements are not stringent and access to LLMs is infrequent. Conversely, local deployment involves installing LLMs on local machines, allowing users to access the models without sending prompts remotely. This method offers greater flexibility for finetuning and mitigates the risks of potential

data leakage. Fortunately, the rapid development of open-source LLMs provides users with the flexibility to deploy these models locally. Open-source LLMs like Qwen 2.5[55] and Llama 3.1[20] have demonstrated comparable performance to leading closed-source commercial LLMs (e.g., OpenAI, Claude) across various well-known benchmarks, such as LiveBench[67], ChatbotArena[11]. Users can also fine-tune these models with domain data, enhancing their comprehension and performance in specific domains.

6 Experimental Evaluation

6.1 Experimental Setup

Datasets. We collected 14 public datasets including 8 classification (C) datasets and 6 regression (R) datasets from UCI repository [63], OpenML [52] and Kaggle [30]. The number of rows in these 14 datasets ranges from 100 to 1M. The detailed statistics and descriptions of these 14 datasets are summarized in Appendix A. We randomly sampled 1000 rows from each dataset as a sample dataset, and partitioned each of them into an 80% training set and a 20% testing set.

Tasks and Metrics. To assess the effectiveness and the efficiency of Adda, we employed five downstream ML models for each dataset from scikit-learn[40]. These models include Decision Tree, Random Forest, Extra Tree, LightGBM and XGBoost, which are adept at handling both classification and regression tasks. Additionally, to evaluate the effectiveness of Adda, we used the area under the ROC curve(AUC) as the performance metric for classification tasks, and $1 - rae$ [58] for regression tasks. Two example ML analytic tasks described in natural language are 'Using the XGBoost model and the ROC metric to predict whether a patient has 10-year risk of coronary heart in the future' and 'Using the Random Forest model and the $1 - rae$ metric to predict the price of each house'. More details are documented in [5]. To evaluate the efficiency of Adda, we measured (1) the end-to-end latency, (2) the time for feature engineering, and (3) the time for model training or prediction. A maximum time limit of 48 hours was set for all experiments.

Adda. We implemented Adda on PostgreSQL. The OpenAI's GPT-4o-latest was used in our two agents, accessed via an API as a paid service. To encode the description of each feature into a high-dimension vector, we offline fine tuned the Qwen2-GTE-1.5B[4] model using the contrastive learning. We set the default parameters for Adda as follows. The sampled data size is 2000. The dataset was split into an 80% training set and a 20% testing set. $w = \frac{\sqrt{2}}{1000}$. $t = 6$ and $s = 10$.

Baselines. We compared Adda against baselines from two domains: AutoFE and the in-database ML. The state-of-the-art (SOTA) AutoFE baselines used in our experiments are as follows. (1) AutoFeat [26] used the expand-and-reduce method for automatic feature engineering, and evaluated each feature using the iterative linear regression model. We used its default parameters. (2) CAAFE [24] sequentially generated new features using an LLM and added features providing performance improvement to current feature set. (3) SmartFeat [46] generated new features using an LLM to select one operator from a pre-defined set of operators, and added them into the feature set without feature evaluation. Both CAAFE and SmartFeat was implemented on GPT-4o-latest. We set ten feature generation iterations for both of them. (4) O1-preview (OpenAI o1-preview) is the current state-of-the-art LLM for logic and math tasks. For an ML analytics task, we conducted one single-turn dialogue with O1-preview to generate a set of features for the task. (5) Raw conducted the downstream algorithms directly on the raw dataset without feature engineering.

The SOTA in-database ML baselines used in our experiments are as follows. (6) Madlib [23], a library including mathematical, statistical and ML methods for in-database analytics. It integrated complex algorithms by registering C-based UDFs. (7) PGML (PostgresML) [41] integrated ML algorithms, such as RandomForest, DecisionTree, by registering rust-based UDFs. Both Madlib and PGML are implemented on PostgreSQL.

Environment. We conducted experiments on a server equipped with an X86 CPU (Xeon(R) Silver 4114, 20 cores @ 2.20GHz), 78GB RAM, a 2TB disk with a maximum transfer rate of 12Gb/s, and an Nvidia Quadro RTX 6000 GPU with 24GB VRAM.

Table 3: Prediction performance of Adda and AutoFE methods (maximum/average score in five downstream algorithms)

	Raw	AutoFeat	CAAFE	O1-preview	SmartFeat	Adda
Heart	72.46/66.40	73.12/67.78	72.55/67.33	73.14/67.06	73.36/66.66	**73.93/68.32**
Bank	94.81/89.61	94.48/89.57	94.84/89.79	94.82/90.11	94.73/90.02	**94.83/90.13**
Adult	79.56/73.15	78.77/73.21	79.00/73.89	80.06/74.18	81.35/75.69	**83.25/76.11**
Titanic	45.54/38.01	48.54/43.99	53.81/40.23	55.35/40.30	83.61/81.42	**84.22/81.54**
Diabetes	86.41/79.85	86.98/81.95	85.60/80.44	**87.04**/80.71	85.82/80.21	86.68/**82.60**
Barpass	84.48/79.73	84.58/79.09	84.42/79.49	84.95/80.43	84.68/**80.22**	**85.09**/80.32
labor	56.05/44.91	> 48 hours	55.10/41.67	54.38/45.24	99.54/98.16	**99.54/98.16**
hepatitis	92.14/87.62	> 48 hours	91.57/87.30	89.32/84.50	91.95/87.84	**98.14/95.10**
bike	99.37/98.70	99.34/98.69	99.32/98.72	99.36/98.68	99.26/98.58	**99.49/98.87**
abalone	43.35/36.03	44.68/34.84	43.35/36.03	44.44/36.18	**46.26/40.47**	**45.23/37.35**
Boston	57.22/54.86	56.35/53.96	56.68/52.32	57.94/53.72	57.04/53.19	**58.58/55.04**
airfoil	81.38/76.65	80.41/76.69	81.38/76.65	82.80/77.09	80.24/74.41	**82.86/77.65**
HouseSale	72.44/68.65	71.81/68.33	72.12/68.84	72.50/68.84	71.96/68.31	**72.52/69.30**
medical	89.44/88.30	89.54/88.33	89.53/88.41	88.53/88.33	89.26/88.17	**89.59/88.62**

6.2 Model Performance Improvement of Adda

To better understand the performance improvement of ML model predictions using Adda, we applied the AutoFE methods SmartFeat, CAAFE, AutoFeat and one SOTA model O1-preview for feature generation, across the 14 datasets with five downstream machine learning algorithms for each. We used AUC and $1 - rae$ to evaluate the accuracy of the generated models and recorded the maximum and average performance across the five downstream algorithms. In Table 3, the best-performing method for each dataset is highlighted in bold.

In Table 3, Adda outperformed all the AutoFE methods significantly. For classification datasets, Adda achieved a performance improvement of maximum and average AUC for up to 84.9% and 114.5% respectively on `Titanic` dataset, while for regression datasets, Adda showed performance enhancement of maximum and average $1 - rae$ for up to 4.3% and 3.7% respectively on `abalone` dataset compared to Raw baseline. Additionally, while other baselines experienced performance regressions on certain datasets comparing to Raw, Adda achieved stable performance improvements across all datasets. The performance degradation in other baselines can be attributed to various factors. In SmartFeat and O1-preview, this could be due to the lack of validation process for newly generated features. In CAAFE this may caused from the limitation in its search space for

its single exploration path. In AutoFeat this may due to the neglect of tasks' information when generating new features. It should be noted that our Adda can still extract effective features from datasets with low column numbers. For example, Adda achieved over 1% performance improvement on the `airfoil` dataset, which only has five columns. We observed that a dataset demonstrating limited performance improvement with Adda showed limited performance improvement with other methods as well.

Moreover, to emphasize the importance of feature engineering in the In-DB scenario, we compared Adda with Madlib and PGML which support In-DB ML. Due to the limited support of downstream algorithm in Madlib and PGML, we chose two downstream algorithm Decision Tree and Random Forest from the original five which supported by both system. As shown in Table 4, the performance of two baselines without feature engineering, was both similar to the raw data performance and showed a significant gap compared to Adda. For classification datasets, Adda achieved maximum and average *AUC* improvement for up to 33.2% and 24.3% respectively on `Heart` dataset, while for regression datasets, Adda achieved performance enhancement of maximum and average $1 - rae$ by up to 4.8% and 8.5% on `Boston` dataset comparing to two baselines.

Table 4: Prediction performance of Adda and In-DB methods(maximum/average score in two downstream algorithms)

Dataset	Madlib	PGML	Adda
Heart	55.53/52.76	55.67/52.43	**73.94/65.59**
Bank	74.70/67.87	73.12/69.07	**94.57/83.63**
Adult	78.42/76.54	77.47/75.23	**83.26/71.77**
Titanic	72.28/71.55	73.34/71.62	**81.34/73.62**
Diabetes	74.13/74.02	74.63/69.20	**86.33/80.66**
Barpass	78.57/77.53	77.81/73.71	**85.10/75.39**
labor	OOM	OOM	**99.16/96.25**
hepatitis	OOM	OOM	**97.14/95.04**
bike	OOM	99.15/98.27	**99.16/98.53**
abalone	34.08/31.34	39.51/32.01	**40.22/32.12**
Boston	46.32/45.72	55.93/47.39	**58.58/51.42**
airfoil	60.66/54.10	77.61/68.48	**77.91/72.63**
HouseSale	OOM	71.29/63.67	**71.51/64.91**
medical	OOM	OOM	**89.11/87.40**

6.3 End-to-End Time Reduction of Adda

To better understand the efficiency of Adda in terms of runtime, especially on large-scale datasets, we compared Adda with other systems using three time metrics mentioned before.

6.3.1 End-to-End latency(min) comparison with AutoFE method. Figure 6 represents the end to end latency for each AutoFE method across five downstream algorithms runs across 12 of 14 datasets. We observed that for smaller data sizes(e.g., Heart, Barpass), the latency time for the three LLM-based AutoFE methods is nearly the identical, with Adda slightly lagging behind SmartFeat and CAAFE due to more communication rounds with the agents. However, CAAFE needed to validate the performance impact of each newly generated feature and SmartFeat must ensure that the code for new feature generation are executable both across the entire dataset. Consequently, the overhead for generating a single feature in both methods was linearly related to the size of dataset, whereas the sampling method in Adda avoided this costly overhead. As a result,

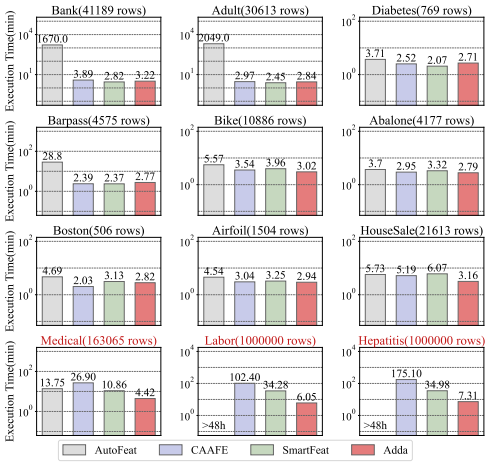

Figure 6: End to End latency(min) of AutoFE methods

on larger datasets with nearly million rows, such as Medical, Labor and Hepatitis, Adda could complete ML workflow in one-tenth the time required by the SmartFeat and CAAFE.

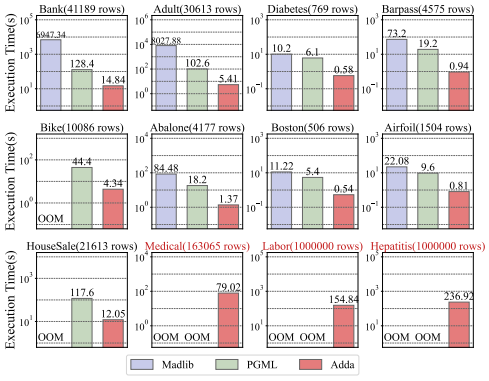

Figure 7: Model Training time(s) of In-DB methods

6.3.2 Model training and prediction time(s) comparison with Madlib and PGML. Figure 7 demonstrated the average time of model training for Adda, Madlib and PGML over two downstream models: Decision Tree and RandomForest. Notably, there was no feature generation SQL statements in Madlib and PGML, which only performed model generation and prediction. We found that Madlib, implemented in SQL and C, was the slowest across all datasets. This was due to SQL's limitations in implementing complex algorithms and its lack of robust support for vectorized parallel execution. Meanwhile, Adda showed roughly a 10× performance advantage over PGML, due to the performance disparity between PGML's Rust library and the more optimized Python library sklearn.

6.3.3 One example for analysis two components in end-to-end Time. As shown in Table 5, we conducted a detailed test of Adda and other AutoFE methods on the Medical dataset, examining the time taken for each components. We observed that Adda achieved up to 8× acceleration in feature engineering time. While there was no significant difference in model training time across these methods

Table 5: Detailed execution time(min) of Adda and other AutoFE methods on Medical Dataset

	AutoFeat	CAAFE	SmartFeat	Adda
Feature Engineering Time	10.45	25.6	9.75	3.32
Model Training Time	3.30	1.13	1.11	1.10
End to End Time	13.75	26.9	10.86	4.42

Table 6: Prediction performance comparison of Adda, Adda-!FD and Adda-!RAG.(maximum/average score)

Dataset	Raw	Adda-!FD	Adda-!RAG	Adda
Heart	68.18/68.18	71.23/69.49	71.04/70.47	**71.23/71.22**
Bank	94.81/94.81	95.21/95.14	95.21/95.17	**95.28/95.20**
Adult	79.02/79.02	93.34/93.32	93.34/93.31	**93.34/93.32**
Titanic	34.41/34.41	84.24/83.50	84.40/83.94	**84.80/84.14**
Diabetes	81.59/81.59	84.80/84.34	85.30/84.92	**86.74/85.36**
Barpass	84.42/84.42	85.28/85.16	85.32/85.06	**85.48/85.37**
labor	53.61/53.61	99.48/99.48	99.48/99.48	**99.48/99.48**
hepatitis	91.26/91.26	98.02/98.01	98.02/98.02	**98.02/98.02**
bike	98.73/98.73	99.51/99.51	99.68/99.52	**99.71/99.53**
abalone	43.36/43.36	45.76/45.71	46.23/46.13	**46.93/46.35**
Boston	56.98/56.98	57.23/56.63	58.13/57.64	**58.33/57.86**
airfoil	76.92/76.92	79.94/79.85	80.26/80.06	**80.96/80.42**
HouseSale	72.44/72.44	72.49/72.40	72.95/72.62	**73.79/72.77**
medical	89.44/89.44	90.02/90.02	90.06/90.04	**90.16/90.08**

for large datasets. Since time for feature engineering accounted for the majority of the end-to-end latency, Adda ultimately achieved up to 6× end-to-end latency reduction.

6.4 Effectiveness of Adda Components

We evaluated the effectiveness of different components in Adda using four variants: Adda-!RAG, Adda-!FD, Adda-!UO and Adda-!CTE with RAG, feature decomposition, UDF Optimization and CTE disabled, respectively.

6.4.1 Components for ML Model Performance Improvement. As shown in Table 6, we conducted accuracy tests on Adda-!FD and Adda-!RAG using all 14 datasets with LightGBM[34] as downstream algorithm and executed each variant three times, summarizing both the maximum and average values. Compared to Adda, the performance of both Adda-!FD and Adda-!RAG declined across all datasets to varying degrees. For classification datasets, Adda-!FD experienced a degradation in maximum and average *AUC* by up to 2.3% and 2.5% respectively, while in regression datasets, the decline of maximum and average $1 - rae$ was up to 1.8% and 2.2% comparing to Adda. This decline was attributed to the lack of code decomposition, which hampered the generation of higher-order features, ultimately leading to poorer performance. Meanwhile, Adda-!RAG it suffered a decline in maximum *AUC* and $1 - rae$ of up to 1.7% and 1.5% in classification and regression tasks respectively, due to the absense of historical information from dynamic prompt, rendering the newly generated feature more useless.

6.4.2 Components for SQL execution speed. As shown in Figure 8, we selected six datasets and plotted a bar chart of model training time for of Adda with two variants based on the SQL statements generated in section 5.4.1. We observed that Adda-!CTE is the slowest in all cases, as materializing data for each operator incurred additional IO overhead, ranging from 1.25 to 7.36. Additionally, Adda

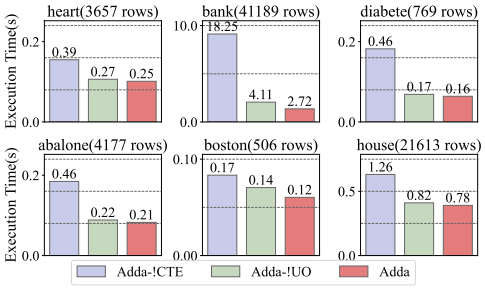

Figure 8: Time for model training comparison of Adda, Adda-!CTE and Adda-!UO.

showed further performance improvements compared to Adda-!UO, with enhancement ranging from 10% to nearly 50% through the optimization of the `Merge` and `Shrink` UDFs.

6.5 Evaluation of parameters in Adda

6.5.1 Influence of w in UCB. As shown in Figure 9(a), we tested the effect of different w values on the final prediction score using the Heart dataset with LightGBM as downstream algorithm with 6 times repeated for each w value, generating box plots based on the prediction scores. We found that as w changes from zero to five, the prediction score initially increased and then showed a decreasing trend. This pattern resulted from w's role in balancing exploration and exploitation, which enhanced performance. The peak average performance occurred at $w = 0.05$. Additionally, we discovered that a larger values of w typically led to greater fluctuations in performance, as they increased the weight of exploration in $\mathcal{U}$.

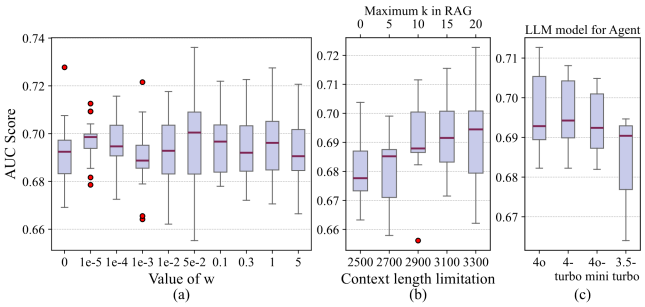

Figure 9: Influence of w, context length and LLM models in Heart dataset

6.5.2 Influence of context length limitation in RAG. In Figure 9(b), we tested the effect of RAG on the prediction score under different context length limitation of the LLM with estimation of the approximate maximum k value in RAG for each context length. We used Heart dataset with 6 times repeated execution with LightGBM, generating box plots. We found that as context length increases, the average prediction accuracy slightly improves about 1%. Meanwhile, we observed that a larger k also leads to an increasement in the volatility of prediction accuracy. This may be because, after incorporating more historical experience, the newly generated features tend to focus more on exploration, as the exploitation information has already been provided by the historical experience.

Table 7: API costs of Adda and other LLM-based baselines.

Execution Cost ($ per task)	O1-preview	CAAFE	SmartFeat	Adda
Openai gpt-4o	0.3	0.02	0.03	0.06
Deepseek-v3	not consider	0.0005	0.0008	0.0015

Table 8: Example features generated in Heart and Adult

Dataset	Feature Name	Feature Description
Heart	avgsb_nonsmok	The average systolic blood pressure for non-smokers
	delta_bp	The delta between systolic and diastolic blood pressure
Adult	edu_hours_ratio	The ratio of study time to work time per week
	avg_child_female	The average number of children for a female

Table 9: The analysis of features generated by Adda and AutoFeat

	Feature Num	Interpretability	Importance		
		SHAP	IG	RFE	FI
AutoFeat	8	28.57%	42.85%	42.85%	28.57%
Adda	7	**42.85%**	**42.85%**	**57.14%**	**42.85%**

6.5.3 Influence of different foundation LLMs for Agent. We measured the AUC score of the LightGBM downstream ML model on the Heart dataset with different foundation models including GPT-4o, GPT-4-turbo, GPT-4o-mini, GPT-3.5-turbo. Results were presented in Figure 9(c) using whisker boxes. We observed that model performance improved (GPT-4o > GPT-4-turbo > GPT-4o-mini > GPT-3.5-turbo) compared to Raw, and powerful LLMs might generated unstable performance features.

6.6 API costs and Feature analysis

6.6.1 API costs of different LLM-based methods. We calculated the API costs by counting the average number of input and output tokens used during API calls. Table 7 presented the API costs per ML analytic tasks for Adda, CAAFE, SmartFeat and O1-preview using gpt-4o and deepseek-v3 [12] as the foundation LLMs, respectively. We observed that O1-preview incurred the highest costs compared to other methods. Adda had twice cost overhead compared to CAAFE and SmartFeat because of the additional overhead brought by the optimization agent. However, the API costs were negligible when deployed locally. Additionally, we measured the time costs of different LLMs and found that their time costs are correlated with the number of tokens. Specifically, more tokens result in longer processing times.

6.6.2 Feature importance analysis. We evaluated the interpretability of generated features using the SHAP [48] metric, and the importance to downstream ML models using three metrics: the information gain (IG), Recursive Feature Elimination (RFE), and Feature Importance (FI). We measured these metrics on the Titanic dataset using Adda and AutoFeat. Table 9 presented the percentage of new generated feature among the top-7 features using these metrics. We observed that features generated by Adda exhibited higher quality and interpretability. Specifically, although Adda generated fewer features, it outperformed AutoFeat by up to 2× across all metrics. Additionally, we demonstrated some generated features for Heart and Adult datasets in Table 8.

Table 10: Prediction performance of Adda and other AutoML methods(maximum/average score)

	Raw	AutoSK	AutoGL	Adda
Heart	68.18/68.18	52.84/52.28	56.24/52.21	**71.23/71.22**
Bank	94.81/94.81	75.85/67.15	76.64/76.64	**95.28/95.20**
Adult	79.02/79.02	80.29/79.95	80.0 5/80.05	**93.34/93.32**
Titanic	34.41/34.41	79.84/79.25	77.88/77.88	**84.80/84.14**
Diabetes	81.59/81.59	73.43/73.26	76.41/74.01	**86.74/85.36**
Barpass	84.42/84.42	77.81/77.59	78.25/78.25	**85.48/85.37**
labor	53.61/53.61	93.42/93.42	96.60/96.60	**99.48/99.48**
hepatitis	91.26/91.26	75.37/75.37	91.03/91.03	**98.02/98.02**
bike	98.73/98.73	**99.99/99.99**	99.24/99.24	99.51/99.51
abalone	43.36/43.36	46.09/46.09	36.66/36.66	**46.93/46.35**
Boston	56.98/56.98	54.37/54.37	54.95/54.95	**58.33/57.86**
airfoil	76.92/76.92	76.90/76.90	**84.99/84.99**	79.96/79.42
HouseSale	72.44/72.44	72.29/72.29	72.77/72.77	**72.79/72.62**
medical	89.44/89.44	88.98/88.98	**91.13/91.13**	90.16/90.08

6.7 Comparison with AutoML Methods

To evaluate the prediction performance of Adda, we introduced two AutoML methods focusing on hyperparameter searching and model selection. (1) AutoSklearn [18] based on Bayesian optimization which leveraging the advantages in meta-learning and ensemble construction (2) AutoGluon [16] ensembled multiple models and stacked them in multiple layers. In this experiment, the evaluation metrics were the same as in Section 5.4.1. As shown in Table 10, we found that Adda maintained a significant advantage, particularly in classification tasks where AutoML methods exhibited unstable performance, even led to performance degradation of over 20% comparing to Raw baseline in Bank and Hepatitis. Ultimately, in 11 out of 14 datasets, Adda outperformed AutoSklearn and AutoGluon, achieving enhancements in maximum AUC and $1 - rae$ of up to 24.3% and 7.3% in classification and regression tasks respectively.

6.8 Multi-client Prediction Serving Performance

To better understand how databases' native capability enhanced specific scenarios in ML workflows, such as efficiency in serving multiple clients for prediction scenarios, we trained the LightGBM on three platforms that can manage models: Adda, PGML, and a Python platform. We deployed several clients using ab [62](python) and pgbench(Adda, PGML) to simulate concurrent client requests in real-world scenarios. For a fair comparison, we store the data on Redis [56] instead of preloading data into memory when using Python. In the experiments below, we tested the throughput, latency of requests, and server memory usage for each method.

6.8.1 Throughput and Latency. As shown in Figure 10(a), when the number of clients was low, Adda and PGML achieved approximately 4 to 6 times higher throughput compared to Python. However, as the number of clients approaches the number of CPU cores, the throughput of PGML rapidly declined, while Adda and Python maintained performance levels. The initial performance gap is due to the in-DB architecture of Adda and PGML which avoided the costly serialization and deserialization overheads caused by database and HTTP. The decline in performance of PGML in the later stages could be attributed to the allocation of more resources internally for each client compared to Adda's pure SQL approach. This could result in heavier overhead for switching as the number

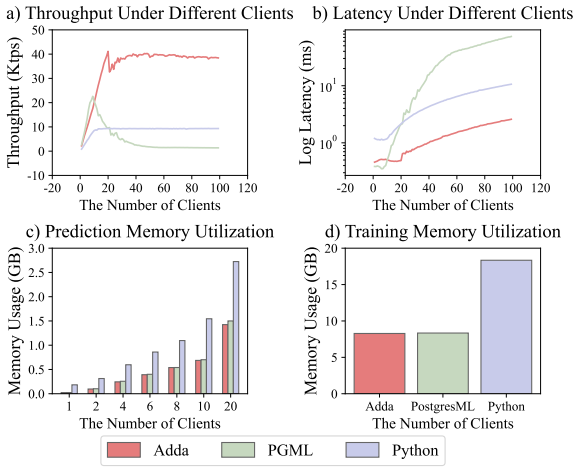

Figure 10: Performance in Multi-client Prediction

of clients increases, leading to a decrease in throughput. Similar to throughput, Adda outperformed Python by a factor of 4 to 6, exhibiting similar latency to PGML at initially but gradually gains over 40× improvement as shown in Figure 10(b).

6.8.2 memory utilization. As shown in Figure 10(c), during multi-client prediction, both Adda and PGML exhibited similar memory overhead as both are developed as upper-level plugins based on PostgreSQL' memory management, reusing shared_buffers and OS page cache to store rows for inference, requiring minimal to no memory allocation for query processing. In contrast, Python incur 3 to 10 times more memory overhead (excluding Redis), primarily due to Python's higher memory usage compared to optimized languages. Meanwhile, as shown in Figure 10(d), during training, Adda and PGML also exhibited nearly half the memory consumption compared to Microservices. This is because PostgreSQL and their plugins share their RAM, whereas pandas, the commonly used library in python for ML required loading the entire data into memory for preprocessing, resulting in additional overhead.

7 Relate work

Large Language Models (LLMs). LLMs, such as BERT [13], GPT-3 [8], and GPT-4 [1], are neural networks that contain billions of parameters and are pre-trained on massive text data. They have demonstrated remarkable foundational capabilities for various tasks, such as text generation [19], question answering [31], complex reasoning [71], and code generation [25]. Recently, researchers have also explored the potential of applying LLMs in data management. For example, [50] casts five data cleaning and integration tasks as prompting tasks to explore the ability of LLMs on these tasks. DITTO [43] formulates entity matching as a sequence-pair classification problem to leverage pre-trained transformer-based LLMs. RPT [61] adopts a transformer-based neural translation architecture for data preparation tasks, such as data cleaning, auto-completion, and schema matching. Retclean [3] demonstrates that LLMs can assist in data cleaning, especially in handling model errors, unseen datasets, and users' private data. Unlike the above works integrating LLMs into the data management processes by serializing data entries and predicting masked tokens, our Adda leverages LLMs to construct features and generate their corresponding code.

Automated Feature Engineering (AutoFE). AutoFE refers to the process of automatically generating suitable features from a dataset to improve predictive learning performance [24, 37, 42]. Various approaches from different perspectives have been studied. For example, DFS [32] and OneBM [39] are based on the *expansion-selection* framework, which generates an exhaustive feature pool followed by feature selection. However, these *expansion-selection* methods incur high computational costs [51]. Other approaches, such as Cognito [37] and AutoFeat [26] based on *iterative-exploration*, address some limitations but have low search efficiency [46]. Subsequently, some *learning-based* methods have also been explored. LFE [51] and ExploreKit [33] leverage a meta-learning approach, while TransGraph [36] is based on Q-learning. Despite these advancements, none of them harness semantic information and domain knowledge in an automated manner. CAAFE [24] and SmartFeat [46] tend to do so by embracing the power of LLMs. CAAFE employs an interactive Chain-of-Thoughts [66] methodology and coarse-grained prompt templates, resulting in a huge search space. SmartFeat defines an operator-based search for generating new features. Unlike CAAFE [24] and SmartFeat [46], our Adda explores their dependency and utilizes LLMs generating a subset of features at a time.

ML in the Database. Improving ML support inside a database has been a recent research focus of the data management community [17, 23, 29, 35, 45, 53, 57, 70]. These works extend the SQL query engine to better support ML either by deploying ML in a database through UDFs or by implementing ML using pure SQL inside a database. For example, MadLib [23], Vertica-ML [17], DB4ML [29], CORE [70] and Raven [53] conduct the deployment of ML training or inference tasks inside a database through UDFs. AC/DC [35], LMFAO [57], DL2SQL [45] implement ML tasks using pure SQL. For unstructured datasets queries, Focus [27] indexes video data using a specialized neural network to detect objects and clusters them to avoid redundant computation. CORE [70] optimize ML inference queries by injecting lightweight proxy models to filter out irrelevant data. For time-series applications, tspDB [2] integrates prediction functionality on top of databases to achieve real-time prediction. Unlike the above approaches, our Adda aims at an end-to-end solution, which automatically converts natural language ML tasks into SQL and UDFs by embracing the power of LLMs.

8 Conclusion

Feature extraction plays a crucial role in ML analytics tasks, yet few in-DBMS tools prioritize this aspect while focusing primarily on model building. In this paper, we introduce Adda, an agent-driven in-database feature generation tool that employs a two-stage processing framework to generate high-quality features within databases for ML tasks. Adda integrates two specialized agents into a general search framework, where they both compete and collaborate toward the same objective. To address performance issues, we translate the Python code generated by LLMs into SQL using just-in-time (JIT) compilation and UDF optimization strategies. Comprehensive experimental evaluations across multiple datasets and ML analytics tasks demonstrated that these techniques significantly enhanced ML model performance and reduced the end-to-end latency.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Anish Agarwal, Abdullah Alomar, and Devavrat Shah. 2021. tspDB: Time Series Predict DB. In *Proceedings of the NeurIPS 2020 Competition and Demonstration Track*, Vol. 133. 27–56.

[3] Mohammad Shahmeer Ahmad, Zan Ahmad Naeem, Mohamed Y. Eltabakh, Mourad Ouzzani, and Nan Tang. 2023. RetClean: Retrieval-Based Data Cleaning Using Foundation Models and Data Lakes. *CoRR* abs/2303.16909 (2023).

[4] Alibaba-NLP. [n. d.]. https://huggingface.co/Alibaba-NLP/gte-Qwen2-1.5B-instruct.

[5] Anonymous. [n. d.]. https://anonymous.4open.science/r/Adda-66E6.

[6] Peter Auer. 2002. Using Confidence Bounds for Exploitation-Exploration Tradeoffs. *Journal of Machine Learning Research* 3 (01 2002), 397–422.

[7] Yoshua Bengio, Aaron C. Courville, and Pascal Vincent. 2012. Representation Learning: A Review and New Perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35 (2012), 1798–1828.

[8] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems*.

[9] Miguel Castro, Barbara Liskov, et al. 1999. Practical byzantine fault tolerance. In *OsDI*, Vol. 99. 173–186.

[10] Xiangning Chen, Qingwei Lin, Chuan Luo, Xudong Li, Hongyu Zhang, Yong Xu, Yingnong Dang, Kaixin Sui, Xu Zhang, Bo Qiao, Weiyi Zhang, Wei Wu, Murali Chintalapati, and Dongmei Zhang. 2019. Neural Feature Search: A Neural Architecture for Automated Feature Engineering. 71–80.

[11] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E. Gonzalez, and Ion Stoica. 2024. Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. arXiv:2403.04132 https://arxiv.org/abs/2403.04132

[12] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT*. Association for Computational Linguistics, 4171–4186.

[14] Pedro Domingos. 2012. A few useful things to know about machine learning. *Commun. ACM* 55, 10 (Oct. 2012), 78–87.

[15] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. arXiv:2305.14325 [cs.CL]

[16] Nick Erickson, Jonas Mueller, Alexander Shirkov, Hang Zhang, Pedro Larroy, Mu Li, and Alexander Smola. 2020. AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data. *arXiv preprint arXiv:2003.06505* (2020).

[17] Arash Fard, Anh Le, George Larionov, Waqas Dhillon, and Chuck Bear. 2020. Vertica-ML: Distributed Machine Learning in Vertica Database *(SIGMOD '20)*. 755–768.

[18] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Springenberg, Manuel Blum, and Frank Hutter. 2015. Efficient and Robust Automated Machine Learning. In *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett (Eds.), Vol. 28. Curran Associates, Inc.

[19] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. Enabling Large Language Models to Generate Text with Citations. In *Empirical Methods in Natural Language Processing (EMNLP)*.

[20] Aaron Grattafiori and Abhimanyu Dubey... 2024. The Llama 3 Herd of Models. arXiv:2407.21783 https://arxiv.org/abs/2407.21783

[21] Sungwon Han, Jinsung Yoon, Sercan Ö. Arik, and Tomas Pfister. 2024. Large Language Models Can Automatically Engineer Features for Few-Shot Tabular Learning. In *ICML*. https://openreview.net/forum?id=fRG45xL1WT

[22] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David A. Sontag. 2022. TabLLM: Few-shot Classification of Tabular Data with Large Language Models. *ArXiv* abs/2210.10723 (2022). https://api.semanticscholar.org/CorpusID:252992811

[23] Joseph M. Hellerstein, Christoper Ré, Florian Schoppmann, Daisy Zhe Wang, Eugene Fratkin, Aleksander Gorajek, Kee Siong Ng, Caleb Welton, Xixuan Feng, Kun Li, and Arun Kumar. 2012. The MADlib analytics library: or MAD skills, the SQL. *Proc. VLDB Endow.* 5, 12 (2012), 1700–1711.

[24] Noah Hollmann, Samuel Müller, and Frank Hutter. 2023. Large Language Models for Automated Data Science: Introducing CAAFE for Context-Aware Automated Feature Engineering. In *Thirty-seventh Conference on Neural Information Processing Systems*.

[25] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, and Chenglin Wu. 2023. MetaGPT: Meta Programming for Multi-Agent Collaborative Framework. *CoRR* abs/2308.00352 (2023).

[26] F. Horn, Robert T. Pack, and Michael Rieger. 2019. The autofeat Python Library for Automated Feature Engineering and Selection. In *PKDD/ECML Workshops*.

[27] Kevin Hsieh, Ganesh Ananthanarayanan, Peter Bodik, Shivaram Venkataraman, Paramvir Bahl, Matthai Philipose, Phillip B. Gibbons, and Onur Mutlu. 2018. Focus: Querying Large Video Datasets with Low Latency and Low Cost. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 269–286.

[28] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. 2019. *Automated Machine Learning: Methods, Systems, Challenges* (1st ed.). Springer Publishing Company, Incorporated.

[29] Matthias Jasny, Tobias Ziegler, Tim Kraska, Uwe Roehm, and Carsten Binnig. 2020. DB4ML - An In-Memory Database Kernel with Machine Learning Support *(SIGMOD '20)*. 159–173.

[30] Kaggle. [n. d.]. https://www.kaggle.com/.

[31] Ehsan Kamalloo, Nouha Dziri, Charles Clarke, and Davood Rafiei. 2023. Evaluating Open-Domain Question Answering in the Era of Large Language Models. 5591–5606.

[32] James Kanter and Kalyan Veeramachaneni. 2015. Deep feature synthesis: Towards automating data science endeavors. 1–10.

[33] Gilad Katz, Eui Chul Richard Shin, and Dawn Xiaodong Song. 2016. ExploreKit: Automatic Feature Generation and Selection. *2016 IEEE 16th International Conference on Data Mining (ICDM)* (2016), 979–984.

[34] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc.

[35] Mahmoud Abo Khamis, Hung Q. Ngo, XuanLong Nguyen, Dan Olteanu, and Maximilian Schleich. 2018. AC/DC: In-Database Learning Thunderstruck *(DEEM'18)*. Article 8, 10 pages.

[36] Udayan Khurana, Horst Samulowitz, and Surya Deepak Turaga. 2017. Feature Engineering for Predictive Modeling Using Reinforcement Learning. *Proceedings of the AAAI Conference on Artificial Intelligence* 32 (09 2017).

[37] Udayan Khurana, Deepak Turaga, Horst Samulowitz, and Srinivasan Parthasrathy. 2016. Cognito: Automated Feature Engineering for Supervised Learning. In *2016 IEEE 16th International Conference on Data Mining Workshops (ICDMW)*. 1304–1307.

[38] Philip Klein, Srikanta Tirthapura, Daniel Sharvit, and Ben Kimia. 2000. A tree-edit-distance algorithm for comparing simple, closed shapes. Society for Industrial and Applied Mathematics, USA.

[39] Hoang Thanh Lam, Beat Buesser, Hong Min, Tran Ngoc Minh, Martin Wistuba, Udayan Khurana, Gregory Bramble, Theodoros Salonidis, Dakuo Wang, and Horst Samulowitz. 2021. Automated Data Science for Relational Data. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. 2689–2692.

[40] Scikit learn Developers. [n. d.]. https://scikit-learn.org/stable/supervised_learning.html.

[41] Silas Marvin Lev Kokotov, Montana Low et al. 2023. PostgresML. https://github.com/postgresml/postgresml

[42] Liyao Li, Haobo Wang, Liangyu Zha, Qingyi Huang, Sai Wu, Gang Chen, and Junbo Zhao. 2023. Learning a Data-Driven Policy Network for Pre-Training Automated Feature Engineering. In *The Eleventh International Conference on Learning Representations*.

[43] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. 2020. Deep Entity Matching with Pre-Trained Language Models. *Proc. VLDB Endow.* 14, 1 (2020), 50–60.

[44] Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. 2023. Towards General Text Embeddings with Multi-stage Contrastive Learning. arXiv:2308.03281 [cs.CL]

[45] Qiuru Lin, Sai Wu, Junbo Zhao, Jian Dai, Feifei Li, and Gang Chen. 2022. A Comparative Study of in-Database Inference Approaches. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. 1794–1807.

[46] Yin Lin, Bolin Ding, H. V. Jagadish, and Jingren Zhou. 2023. SMARTFEAT: Efficient Feature Construction through Feature-Level Foundation Model Interactions. arXiv:2309.07856 [cs.DB]

[47] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023*.

[48] Scott Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. arXiv:1705.07874 [cs.AI] https://arxiv.org/abs/1705.07874
[49] T.J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320.
[50] Avanika Narayan, Ines Chami, Laurel J. Orr, and Christopher Ré. 2022. Can Foundation Models Wrangle Your Data? *Proc. VLDB Endow.* 16, 4 (2022), 738–746.
[51] Fatemeh Nargesian, Horst Samulowitz, Udayan Khurana, Elias B. Khalil, and Deepak Turaga. 2017. Learning Feature Engineering for Classification. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI-17*. 2529–2535.
[52] OpenML. [n. d.]. https://www.openml.org/.
[53] Kwanghyun Park, Karla Saur, Dalitso Banda, Rathijit Sen, Matteo Interlandi, and Konstantinos Karanasos. 2022. End-to-end Optimization of Machine Learning Prediction Queries *(SIGMOD '22)*. 587–601.
[54] Jignesh M. Patel, Michael J. Carey, and Mary K. Vernon. 1994. Accurate modeling of the hybrid hash join algorithm *(SIGMETRICS '94)*. Association for Computing Machinery, New York, NY, USA, 56–66.
[55] Qwen Team. 2024. Qwen2.5: A Party of Foundation Models. https://qwenlm.github.io/blog/qwen2.5/
[56] Pieter Noordhuis Salvatore Sanfilippo, Oran Agra et al. 2023. Redis. https://github.com/redis/redis
[57] Maximilian Schleich and Dan Olteanu. 2020. LMFAO: An engine for batches of group-by aggregates: layered multiple functional aggregate optimization. *Proc. VLDB Endow.* 13, 12 (aug 2020), 2945–2948.
[58] Maxim Shcherbakov, Adriaan Brebels, N.L. Shcherbakova, Anton Tyukov, T.A. Janovsky, and V.A. Kamaev. 2013. A survey of forecast error measures. *World Applied Sciences Journal* 24 (01 2013), 171–176.
[59] Claudio Spiess, David Gros, Kunal Suresh Pai, Michael Pradel, Md. Rafiqul Islam Rabin, Amin Alipour, Susmit Jha, Prem Devanbu, and Toufique Ahmed. 2024. Calibration and Correctness of Language Models for Code. *CoRR* abs/2402.02047 (2024).
[60] Michael Stonebraker and Lawrence A. Rowe. 1986. The design of POSTGRES. In *Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data* (Washington, D.C., USA) *(SIGMOD '86)*. Association for Computing Machinery, New York, NY, USA, 340–355. https://doi.org/10.1145/16894.16888
[61] Nan Tang, Ju Fan, Fangyi Li, Jianhong Tu, Xiaoyong Du, Guoliang Li, Samuel Madden, and Mourad Ouzzani. 2021. RPT: Relational Pre-trained Transformer Is Almost All You Need towards Democratizing Data Preparation. *Proc. VLDB Endow.* 14, 8 (2021), 1254–1261.
[62] The Apache Software Foundation. 2023. Apache Benchmark. https://httpd.apache.org/docs/2.4/programs/ab.html.
[63] Irvine University of California. [n. d.]. https://archive.ics.uci.edu/.
[64] Aäron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation Learning with Contrastive Predictive Coding. *ArXiv* abs/1807.03748 (2018).
[65] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models.
[66] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 35. 24824–24837.
[67] Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Siddartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. 2024. LiveBench: A Challenging, Contamination-Free LLM Benchmark. arXiv:2406.19314 https://arxiv.org/abs/2406.19314
[68] David Kofoed Wind. 2014. Concepts in predictive machine learning. *Technical University of Denmark* (2014), 1–129.
[69] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2024. C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597 [cs.CL]
[70] Zhihui Yang, Zuozhi Wang, Yicong Huang, Yao Lu, Chen Li, and X. Sean Wang. 2022. Optimizing machine learning inference queries with correlative proxy models. (2022), 2032–2044.
[71] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR*. OpenReview.net.
[72] Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262.
[73] Tianping Zhang, Zheyu Zhang, Zhiyuan Fan, Haoyan Luo, Fengyuan Liu, Qian Liu, Wei Cao, and Jian Li. 2023. OpenFE: Automated Feature Generation with Expert-level Performance. arXiv:2211.12507 [cs.LG]
[74] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models. arXiv:2310.04406 [cs.AI]
[75] Guanghui Zhu, Zhuoer Xu, Chunfeng Yuan, and Yihua Huang. 2022. DIFER: Differentiable Automated Feature Engineering.

A Appendix

In this section, we provide detailed information about each dataset used in the experiments.

Table 11: The statistics and descriptions of the 14 public datasets.

	C/R	Attrs/Rows	Source	Description
Heart	C	14/3657	Kaggle	A dataset contains personal information and health indicators of individuals, e.g., age and gender, blood pressure.
Bank	C	21/41189	Kaggle	A dataset describes customers' statistics at bank, such as amount of loan.
Adult	C	14/30613	Kaggle	A dataset describes individuals' life and work status, such as daily reading time and weekly working hours.
Titanic	C	12/892	Kaggle	Statistics of all passengers on the Titanic, e.g., age, gender, and ticket class.
Diabetes	C	8/769	Kaggle	A dataset contains physical health indicators related to diabetes, such as blood pressure.
Barpass	C	12/4575	Kaggle	Statistics of students who attend the bar exam, such as their grade point averages (GPAs).
labor	C	18/1000000	OpenML	Annual work-related statistics of employees, such as average working hours.
hepatitis	C	20/1000000	OpenML	Physical statistics of hepatitis patients, such as albumin levels.
bike	R	17/10886	Kaggle	Daily usage statistics of bikes, such as the total usage hours.
abalone	R	8/4177	UCI	A dataset includes information such as sex, size, and weight of abalones.
Boston	R	13/506	Kaggle	A dataset contains information like locations and surroundings of houses in Boston.
airfoil	R	5/1504	UCI	A dataset provides physical statistics of airfoils, such as chord length.
HouseSale	R	22/21613	Kaggle	A dataset offers detailed information on houses for sale, including the number of bathrooms and bedrooms.
medical	R	11/163065	OpenML	A dataset describes the available medical resources for individuals, such as total payments.